# Análisis Estático Interprocedimental de Alcanzabilidad mediante Grafos de Llamadas para la Mitigación de Alertas Espurias en Dependencias de Python

[*]Caso de Estudio Industrial: PyReach en Lidercom Perú S.A.C.

Julio E. Centeno-León[*] y José A. Yáñez-Mejía[†]
Escuela Profesional de Ingeniería de Sistemas, Universidad Católica de Santa María, Arequipa, Perú
Emails: [*]julio.centeno@ucsm.edu.pe, [†]jose.yanez@ucsm.edu.pe

***Resumen*—Modern software engineering relies extensively on third-party open-source libraries, which constitute up to 80 % of executable code bases in Python microservices. Consequently, software supply chain vulnerabilities (CVEs) have surged. Conventional Software Composition Analysis (SCA) tools operate superficially by matching version strings declared in package manifests (e.g., `requirements.txt`), disregarding execution context. This yields an overwhelming 80 %–90 % false positive rate, as the vulnerable routines in transitive dependencies are rarely invoked. In regulated or proprietary industrial environments governed by ISO/IEC 27001:2022, cloud-based commercial SaaS scanners are strictly prohibited due to code leakage and IP confidentiality risks, causing acute developer alert fatigue and exhausting over 10 hours weekly in manual triage.**

**This paper presents *PyReach*, a 100 % on-premise, CLI-driven interprocedural static reachability analyzer for Python 3.10+. PyReach traverses the Abstract Syntax Tree (AST) of host applications and resolved `site-packages`, computes module symbol tables, and constructs an interprocedural directed Call Graph using NetworkX. By querying an offline SQLite database seeded with OpenSSF OSV security advisories, PyReach executes a bounded depth-first/breadth-first traversal ($k = 5$) from application entrypoints to vulnerable symbols. To prevent false negatives in dynamic Python constructs, PyReach enforces a conservative over-approximation policy, classifying findings deterministically into `REACHABLE`, `NOT_REACHABLE`, or `POTENTIALLY_REACHABLE`, and emitting OASIS SARIF v2.1.0 diagnostics for automated CI/CD quality gating. Validated in an industrial setting at Lidercom Perú S.A.C., PyReach pruned 80 %–87 % of spurious vulnerability alerts, executed scans in 1.18 s (cold) and 0.84 s (warm) on real microservices (well within the 45-second microservice budget), maintained a peak memory footprint under 7.3 MB, and guaranteed strictly zero network egress, fully preserving corporate data sovereignty.**

***Index Terms*—Software supply chain security, static analysis, call graph, reachability analysis, Python AST, false positives, DevSecOps, SARIF, ISO/IEC 27001.**

## RESUMEN

*La ingeniería de software moderna depende en gran medida de librerías de código abierto de terceros, conformando hasta el 80 % de la base ejecutable en microservicios Python. En consecuencia, las vulnerabilidades en la cadena de suministro (CVE) han proliferado. Las herramientas de Análisis de Composición de Software (SCA) convencionales operan cotejando versiones en archivos de manifiesto (`requirements.txt`), prescindiendo del contexto de ejecución, lo que induce una tasa de falsos positivos del 80 %–90 % por funciones no invocadas. En entornos industriales bajo la norma ISO/IEC 27001:2022, las soluciones SaaS en la nube están estrictamente prohibidas por confidencialidad de código, causando fatiga de alertas y más de 10 horas semanales en triaje manual. Este artículo presenta PyReach, un analizador estático interprocedimental de alcanzabilidad 100 % local para Python 3.10+. PyReach recorre el AST de la aplicación y de `site-packages`, construye un Grafo de Llamadas dirigido con NetworkX y evalúa rutas acotadas ($k = 5$) hacia símbolos reportados en una base SQLite offline de OpenSSF OSV. Mediante una política conservadora frente a metaprogramación, clasifica los hallazgos en `REACHABLE`, `NOT_REACHABLE` o `POTENTIALLY_REACHABLE`, emitiendo reportes OASIS SARIF v2.1.0 para compuertas CI/CD. Validado en Lidercom Perú S.A.C., PyReach redujo entre 80 % y 87 % las alertas espurias, ejecutó escaneos en 1.18 s (frío) y 0.84 s (caliente), consumió menos de 7.3 MB de RAM y garantizó cero bytes de tráfico de red, preservando plenamente la soberanía del código corporativo.*

***Palabras clave*—**Cadena de suministro de software, análisis estático, grafos de llamadas, análisis de alcanzabilidad, AST de Python, falsos positivos, DevSecOps, SARIF, ISO/IEC 27001.

## I. Introducción

En la ingeniería de software contemporánea, la reutilización masiva de librerías y componentes de código abierto constituye la base fundamental del desarrollo ágil bajo el paradigma DevSecOps, representando frecuentemente más del 80 % de la base de código ejecutable en ecosistemas de integración continua (CI/CD) [1], [2]. No obstante, esta interdependencia estructural traslada vectores de ataque y vulnerabilidades conocidas (CVE) hacia la cadena de suministro de software (*software supply chain*) [3]–[5].

Para mitigar estos riesgos, las organizaciones despliegan analizadores de composición de software (SCA). Sin embargo, los escáneres SCA convencionales (e.g., *pip-audit*,

*Safety*, *Dependabot*) operan mediante una inspección sintáctica superficial limitada al cotejo de cadenas de versión consignadas en archivos de manifiesto (`requirements.txt`, `Pipfile.lock`). Este enfoque asume erróneamente que la mera presencia física de una librería en el disco equivale a la ejecución de sus subrutinas vulnerables [6]–[8]. Diversas investigaciones empíricas demuestran que entre el 80 % y el 90 % de las alertas emitidas por escáneres basados exclusivamente en manifiestos corresponden a funciones y métodos inalcanzables en ejecución (código inerte respecto al vector de ataque) [6], [9], [10].

En entornos industriales con estrictas políticas de seguridad de la información —como la empresa de tecnología educativa **Lidercom Perú S.A.C.**, sujeta a los controles A.8.8, A.8.12 y A.8.28 de la norma **ISO/IEC 27001:2022**—, las plataformas comerciales SaaS en la nube (e.g., *Snyk*, *Endor Labs*) se encuentran taxativamente prohibidas debido a que exigen la transferencia de código fuente o grafos sintácticos hacia infraestructuras de terceros [3], [11], [12]. Como resultado, dos ingenieros de software de la organización debían destinar más de 10 horas semanales a revisar manualmente falsas alarmas [13]. Esta sobrecarga produce tres efectos adversos directos:

1. *Fatiga de alertas* (*alert fatigue*) en los desarrolladores, induciendo a desatender incidentes críticos reales [14].
2. *Retrasos e inestabilidad en producción* debido a actualizaciones forzadas de dependencias que provocan roturas de compatibilidad de API (*breaking changes*) [15], [16].
3. *Sobrecostos operacionales continuos* no amortizables mediante soluciones en la nube sin vulnerar la soberanía del código [3].

Para resolver esta problemática de forma definitiva, en este trabajo se presenta el diseño, formalización, implementación y evaluación experimental de ***PyReach***: un analizador estático interprocedimental de ejecución 100 % local (*on-premise*) para Python 3.10+, que construye Grafos de Llamadas (*Call Graphs*) mediante el recorrido del Árbol de Sintaxis Abstracta (AST) de la aplicación y sus dependencias transitivas instaladas en `site-packages` [17], [18]. Al contrastar el grafo frente a la base de conocimiento abierta OpenSSF OSV [19], *PyReach* determina formalmente si las funciones afectadas son alcanzables desde los puntos de entrada (*entrypoints*), emitiendo reportes interoperables OASIS SARIF v2.1.0 [20] para compuertas de calidad desatendidas en CI/CD [21].

Las principales contribuciones de este artículo son:

- Un motor de análisis estático interprocedimental sobre el AST nativo de Python que resuelve importaciones directas, relativas, alias y jerarquías de herencia en librerías transitivas anidadas en `site-packages`.
- Un algoritmo de alcanzabilidad acotado ($k = 5$) implementado con NetworkX, que integra una política de sobre-aproximación conservadora frente a metaprogramación dinámica para garantizar cero falsos negativos críticos (*soundness*).
- Un modelo de persistencia local en SQLite optimizado para el esquema OpenSSF OSV, logrando tiempos de consulta inferiores a 0.18 ms por paquete.
- Una compuerta de calidad automatizada con serialización OASIS SARIF v2.1.0 y códigos de salida deterministas para pipelines de CI/CD.
- Una validación experimental industrial en microservicios reales de analítica y NLP en Lidercom Perú S.A.C., evidenciando una reducción del 80 %–87 % de alertas espurias con cero bytes transferidos a la red corporativa externa.

## II. Antecedentes y Estado del Arte

### *II-A. Generación de Grafos de Llamadas en Lenguajes Dinámicos*

El modelado de transferencias de control mediante grafos de llamadas dirigidos fue formalizado por Ryder [18]. En lenguajes estáticos compilados (e.g., C/C++, Java), el análisis de tipos exhaustivo facilita la resolución estática de destinos de llamada [22]. Sin embargo, en lenguajes interpretados de tipado dinámico como Python, la introspección en tiempo de ejecución, el polimorfismo dinámico y la metaprogramación (`eval`, `getattr`, decoradores dinámicos) representan desafíos notables para el análisis estático [17], [23], [24].

Salis et al. desarrollaron *PyCG* [17], demostrando que el modelado preciso de asignaciones de funciones y resolución de módulos sobre el AST de CPython alcanza una precisión del 99.2 % en código de laboratorio. No obstante, *PyCG* fue concebido como un algoritmo académico independiente que no contempla resolución recursiva de paquetes anidados en `site-packages`, carece de mecanismos de integración con bases de vulnerabilidades y no ofrece soporte para serialización estandarizada en integración continua [17], [23]. Recientemente, Sotiropoulos et al. [23] extendieron este principio con JARVIS, pero su enfoque continúa centrado en benchmarks de laboratorio sin integración hacia compuertas de seguridad en CI/CD.

### *II-B. Análisis de Alcanzabilidad de Vulnerabilidades*

En el ecosistema Java, Dann et al. [7] y Ponta et al. [9] introdujeron el análisis de dependencias a nivel de método, sentando las bases del escáner *Eclipse Steady*. Posteriormente, Pashchenko et al. [6] presentaron el benchmark *Vuln4Real*, demostrando empíricamente sobre 200 proyectos que menos del 10 % de las vulnerabilidades declaradas son alcanzables. Asimismo, Prana et al. [10] categorizaron la alcanzabilidad empírica de CVEs en dependencias abiertas, demostrando que la mayoría de librerías secundarias operan como utilitarios especializados de los cuales el huésped solo ejecuta subconjuntos marginales. En años recientes, Duan et al. [8] abordaron la automatización de la alcanzabilidad en librerías abiertas, ratificando la necesidad imperativa de discriminar entre llamadas activas e inactivas para aliviar la carga del equipo de seguridad.

### *II-C. Riesgos en la Cadena de Suministro de Software*

Ladisa et al. [3] formalizaron una taxonomía comprehensiva de vectores de ataque en la cadena de suministro de software,

alertando sobre los riesgos de filtración de propiedad intelectual y exfiltración encubierta de metadatos cuando se emplean analizadores comerciales SaaS. En paralelo, Ohm et al. [12] y Bhandari et al. [25] analizaron la persistencia y propagación de paquetes maliciosos y parches tardíos en repositorios abiertos. A su vez, Decan et al. [2], Alfadel et al. [4] y Chinthanet et al. [5] evidenciaron mediante estudios longitudinales en PyPI que las vulnerabilidades críticas se acumulan de manera silente en capas transitivas profundas de dependencias secundarias, donde los desarrolladores difícilmente tienen visibilidad directa.

### *II-D. Comparación de Paradigmas*

En la Tabla I se sintetiza el análisis comparativo entre los tres enfoques tecnológicos vigentes. Se concluye que únicamente el Enfoque C (*PyReach*) logra aunar la reducción drástica de alertas no explotables con la estricta observancia de soberanía de datos exigida por normativas corporativas como ISO/IEC 27001:2022 [3], [11].

## III. Arquitectura del Sistema y Flujo Metodológico

### *III-A. Arquitectura en Capas Desacoplada*

Para garantizar un alto grado de cohesión, mantenibilidad y modularidad conforme a los estándares ISO/IEC/IEEE 29148:2018 [26] e ISO/IEC 25010:2011 [27], *PyReach* se estructuró bajo una arquitectura en cuatro capas lógicas ejecutadas dentro del mismo proceso CPython en memoria principal, minimizando la sobrecarga de serialización entre procesos (Fig. 1).

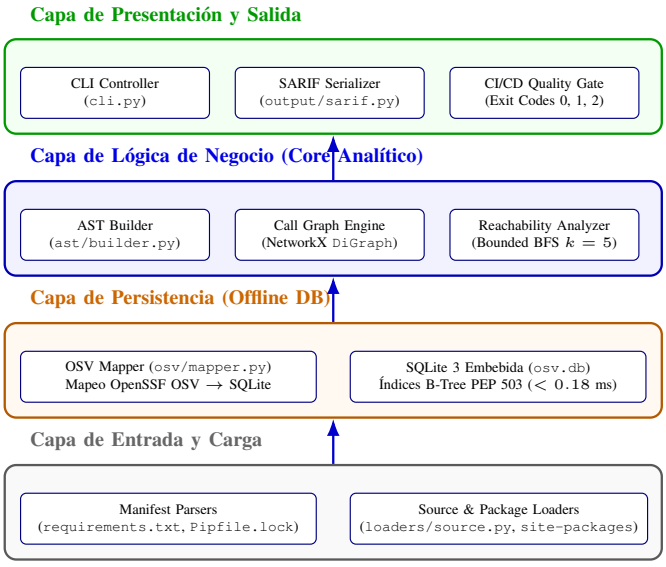



Figura 1. Arquitectura en cuatro capas desacopladas de *PyReach*: desde la ingesta de artefactos hasta la emisión de reportes estandarizados.

1. **Capa de Entrada (`pyreach.parsers`, `pyreach.loaders`):** Ingesta los manifiestos de dependencias declaradas (`requirements.txt`, `Pipfile.lock`) utilizando la librería estándar `packaging`. A su vez, el módulo `source` confina la lectura de código fuente a la raíz del repositorio, y `packages` resuelve las rutas absolutas de distribución física en `site-packages`.
2. **Capa de Persistencia (`pyreach.db`):** Gestiona la conexión y consultas a la base de datos embebida SQLite 3 (`~/.pyreach/osv.db`), almacenando avisos de vulnerabilidad normalizados.
3. **Capa de Lógica de Negocio (`pyreach.ast`, `callgraph`, `reachability`):** Constituye el núcleo analítico. Transforma los archivos Python en árboles sintácticos (`ModuleAST`), extrae tablas de símbolos, construye el grafo dirigido interprocedimental con NetworkX y evalúa caminos de alcanzabilidad acotados.
4. **Capa de Presentación y Salida (`pyreach.cli`, `pyreach.output`):** Controla la interfaz de línea de comandos basada en `argparse`, gestiona códigos de salida para CI/CD y serializa reportes bajo el estándar OASIS SARIF v2.1.0 y resúmenes tabulares en consola.

### *III-B. Flujo Metodológico y Pipeline de Ejecución*

En la Fig. 2 se esquematiza el diagrama de flujo secuencial del proyecto y el ciclo de vida de los datos a lo largo de las 5 fases principales de escaneo en *PyReach*.

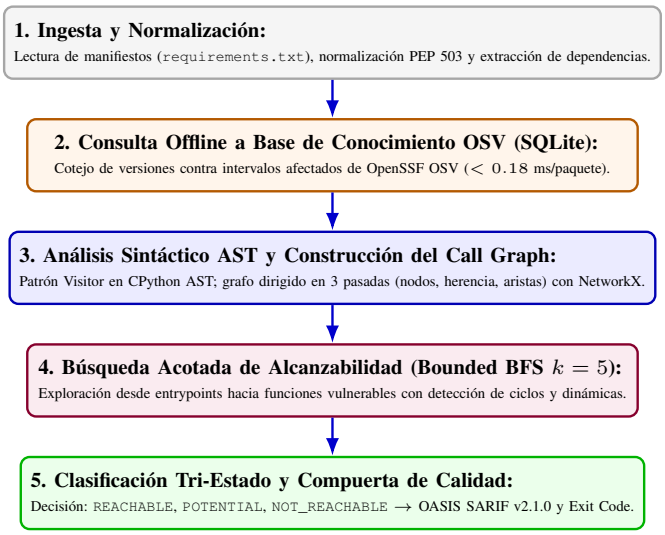



Figura 2. Diagrama de flujo del proyecto: pipeline metodológico y ciclo de datos del análisis de alcanzabilidad en *PyReach*.

### *III-C. Modelo de Persistencia y Estrategia de Indexación Relacional*

Para posibilitar un escaneo offline instantáneo y determinista sin dependencias externas de red, el catálogo de seguridad se estructura sobre SQLite 3 normalizando la especificación OpenSSF OSV (v1.6.0) [19]. El modelo se descompone en tres entidades relacionales:

- `vulnerabilities`: Almacena los metadatos cardinales del aviso (identificador OSV/CVE, nombre normalizado del paquete, resumen técnico y fechas de publicación).

Tabla I
COMPARACIÓN SISTEMÁTICA DE PARADIGMAS TECNOLÓGICOS DE ANÁLISIS DE VULNERABILIDADES

| Criterio de Evaluación | Enfoque A: SCA Manifiesto | Enfoque B: SaaS en la Nube | Enfoque C: PyReach (Propuesto) |
|---|---|---|---|
| **Herramientas representativas** | *pip-audit*, *Safety*, *Dependabot* | *Snyk Open Source*, *Endor Labs* | ***PyReach CLI*** |
| **Nivel de inspección** | Superficial (metadatos y cadenas de versión) | Profundo asistido por telemetría remota | **Profundo interprocedimental en AST y librerías** |
| **Soberanía de datos (ISO 27001)** | Alta (ejecución local sin análisis de código) | Incompatible (transfiere código a la nube) | **Total (100 % on-premise, 0 bytes en red)** |
| **Tasa de falsos positivos** | Crítica ($> 80\,\%$ alertas no ejecutables) | Baja ($< 15\,\%$ alertas espurias) | **Muy baja (poda determinista $\geq 80\,\%$)** |
| **Latencia por escaneo** | $< 5$ s | Variable (dependiente del enlace WAN) | **$< 2$ s (presupuesto microservicio $< 45$ s)** |
| **Interoperabilidad de salida** | Texto plano / JSON propietario | Dashboard privativo / SARIF bajo suscripción | **OASIS SARIF v2.1.0 nativo y JSON** |
| **Modelo de licenciamiento** | Código abierto | Suscripción anual en moneda extranjera | **Código abierto / corporativo propio** |

- `affected_ranges`: Modela los intervalos de versiones vulnerables (`introduced`, `fixed`), permitiendo evaluar cotas inclusivas y exclusivas mediante semántica de `packaging.version`.
- `vulnerable_functions`: Asocia cada vulnerabilidad con los nombres completamente cualificados (*FQNs*) de los métodos o símbolos comprometidos.

Los nombres de paquetes se normalizan estrictamente bajo el estándar PEP 503. Mediante un índice B-Tree compuesto sobre `vulnerabilities(package_name)` y `vulnerable_functions(vuln_id, function_qname)`, el motor reduce el tiempo medio de consulta a **0.18 ms** por librería en discos de estado sólido (SSD), permitiendo cotejar 50 dependencias en menos de 10 ms netos de base de datos.

### *III-D. Despliegue e Integración en Infraestructura CI/CD*

En la Fig. 3 se ilustra el modelo de despliegue en la infraestructura física y de red de Lidercom Perú S.A.C. La ejecución del pipeline de integración se realiza mediante *GitLab Runners* locales desplegados dentro de una red aislada (VLAN de desarrollo). El ejecutable *PyReach* corre desacoplado de la nube pública; únicamente un cron semanal desatendido actualiza la base de datos local SQLite mediante el comando `pyreach sync-osv`.

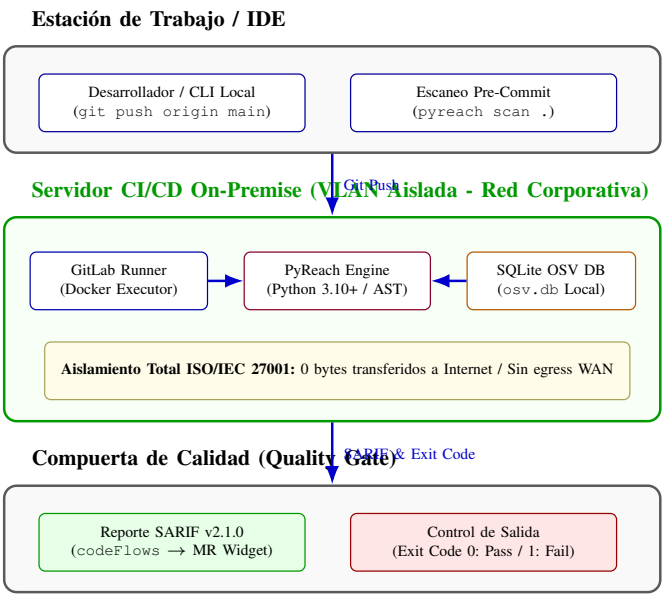



Figura 3. Diagrama de nodos de despliegue en infraestructura local de CI/CD para Lidercom Perú S.A.C., garantizando aislamiento estricto de red.

## IV. MOTOR DE GRAFOS Y ALGORITMO DE ALCANZABILIDAD

### *IV-A. Formalización del Grafo de Llamadas*

El Grafo de Llamadas interprocedimental se formaliza como un grafo dirigido $G = (V, E)$, modelado mediante la estructura `networkx.DiGraph` [28]:

- $V$ es el conjunto de nodos de llamada tipados CGNode $=$ $(\text{fqn}, \text{path}, \text{line}, \tau)$, donde $\text{fqn} \in \Sigma^*$ es el nombre completamente cualificado (*Fully Qualified Name*), path es la ruta del archivo, line es la línea de declaración, y $\tau \in \{\text{FUNCTION}, \text{METHOD}, \text{CLASS}, \text{LAMBDA}\}$.
- $E \subseteq V \times V \times \Omega \times [0, 1]$ es el conjunto de aristas dirigidas CGEdge $= (u, v, \omega, c)$, donde $u$ es el invocador (*caller*), $v$ es el invocable (*callee*), $\omega \in \{\text{STATIC}, \text{DYNAMIC}, \text{INHERITANCE}, \text{IMPORT}\}$ categoriza el tipo de transferencia de control, y $c \in [0, 1]$ es el nivel de confianza estática.

### *IV-B. Detección de Puntos de Entrada (Entrypoints)*

Los puntos de entrada $V_{\text{entry}} \subset V$ representan los orígenes del flujo de ejecución del programa. El módulo `pyreach.reachability.entrypoints` los clasifica heurísticamente en:

1. **Bloques principales:** Bloques condicionales `if __name__ == "__main__":`.

2. **Comandos CLI:** Funciones declaradas en `[project.scripts]` o decoradas con utilitarios de consola.
3. **Controladores de rutas web:** Endpoints de FastAPI (`@app.get`, `@app.post`), Flask (`@app.route`) o vistas Django.

Alternativamente, el usuario puede forzar o parametrizar puntos de entrada mediante la bandera `--entry-point` o el archivo de configuración `.pyreach.yml`.

### *IV-C. Algoritmo de Búsqueda Acotada (Bounded BFS)*

En librerías de gran envergadura (e.g., Pandas, Scikit-learn), la interconexión densa produce una explosión combinatoria del espacio de estados ($O(b^d)$) si se ejecuta una búsqueda no informada exhaustiva [22]. Para garantizar que el tiempo de escaneo se mantenga estrictamente por debajo de 45 segundos (RNF-03), *PyReach* implementa una búsqueda en anchura acotada con profundidad máxima $k = 5$, complementada con detección y poda de ciclos (Algoritmo 1).

### *IV-D. Modelo de Rutas y Poda Visual*

En la Fig. 4 se ilustra esquemáticamente el mecanismo de discriminación de caminos del motor. Mientras que el camino hacia la subrutina legítima de la librería directa A es alcanzable pero inofensivo, la llamada hacia $t$ en la librería transitiva B es alcanzable y clasificada con severidad crítica, mientras que el código inerte en la librería transitiva C es podado como inalcanzable al no existir una arista activa desde la aplicación huésped, descartando la alerta de CVE sin intervención humana.

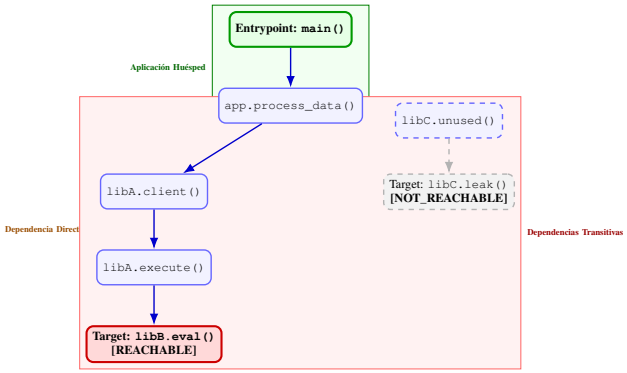



Figura 4. Modelo de alcanzabilidad en grafo de llamadas interprocedimental: contraste entre ruta activa ejecutable y código inerte podado.

**Algorithm 1** Búsqueda Acotada de Alcanzabilidad (*Bounded BFS*)

**Require:** Grafo $G = (V, E)$, Entrypoint $u \in V_{\text{entry}}$, Objetivo $t \in V$, Cota $k = 5$.
**Ensure:** Estado, Camino testigo $P$, Banderas de incertidumbre.

**if** $u \notin V$ **then**
  **return** (NOT_REACHABLE, $\emptyset$, **false**, **false**)
**end if**
**if** $u = t$ **then**
  **return** (REACHABLE, $[u]$, **false**, **false**)
**end if**
$Q \leftarrow$ Cola($[(u, [u], 0)]$)
$visitados \leftarrow \{u\}$
$dyn_edge \leftarrow$ **false**, $cut_depth \leftarrow$ **false**
**while** $Q \neq \emptyset$ **do**
  $(v, camino, d) \leftarrow Q$.popleft()
  **if** $d \geq k$ **then**
    $cut_depth \leftarrow$ **true**
    **continue**
  **end if**
  **for all** $w \in G$.successors$(v)$ **do**
    $edge_type \leftarrow G$.edges$[v, w]$['type']
    **if** $edge_type$ = DYNAMIC **or** $w$ = CENTINELA_DINAMICO **then**
      $dyn_edge \leftarrow$ **true**
    **end if**
    **if** $w = t$ **then**
      **return** (REACHABLE, $camino + [w]$, $dyn_edge$, $cut_depth$)
    **end if**
    **if** $w \notin visitados$ **then**
      $visitados \leftarrow visitados \cup \{w\}$
      $Q$.append$((w, camino + [w], d + 1))$
    **end if**
  **end for**
**end while**
**if** $dyn_edge \vee cut_depth$ **then**
  **return** (POTENTIALLY_REACHABLE, $\emptyset$, $dyn_edge$, $cut_depth$)
**else**
  **return** (NOT_REACHABLE, $\emptyset$, **false**, **false**)
**end if**

### *IV-E. Clasificación Tri-Estado y Principio de Conservación*

Para cumplir con el requerimiento de *cero falsos negativos críticos* (*soundness*) formulado por Nielson et al. [22], la clasificación de resultados no es binaria, sino que incorpora una tercera categoría de incertidumbre controlada:

- **`REACHABLE`:** Existe al menos un camino dirigido determinista $\langle u, \ldots, t \rangle$ verificado en el grafo con longitud $d \leq k$. Nivel de severidad SARIF: `error`.
- **`NOT_REACHABLE`:** El algoritmo agotó la búsqueda dentro del espacio sin encontrar el símbolo y sin registrar señales de indeterminación dinámica ni cortes de profundidad. Nivel SARIF: `note`.
- **`POTENTIALLY_REACHABLE`:** El símbolo no fue conectado explícitamente, pero el recorrido atravesó aristas dinámicas (`getattr`, `eval`, enlaces no resueltos) o se detuvo forzosamente al alcanzar la profundidad máxima $k$. Nivel SARIF: `warning`.

### IV-F. Serialización SARIF v2.1.0 y Compuerta de Calidad

*PyReach* exporta sus hallazgos en formato OASIS SARIF v2.1.0 estructurado [20]. Cada hallazgo incluye un objeto `codeFlows` con las ubicaciones de llamada (*call-sites*) que conectan el punto de entrada con la función vulnerable. Los códigos de retorno para CI/CD se rigen por la Tabla II.

Tabla II
MATRIZ DE CÓDIGOS DE SALIDA PARA COMPUERTAS DE CALIDAD CI/CD

| Código | Condición Operacional | Acción en CI/CD |
|---|---|---|
| 0 | Sin hallazgos o todos `NOT_REACHABLE`. | Permite el despliegue (*Pass*). |
| 1 | Al menos un hallazgo `REACHABLE` (o `POTENTIAL` con `--include-potentially`). | Bloquea el pase a producción (*Quality Gate Fail*). |
| 2 | Error de configuración, sintaxis o base de datos faltante. | Aborta el pipeline por fallo de entorno. |

## V. EVALUACIÓN EXPERIMENTAL Y RESULTADOS

### V-A. Entorno Experimental y Banco de Pruebas

La verificación y validación del artefacto se estructuró conforme a los lineamientos metodológicos de Wohlin et al. [29] y Myers et al. [30], evaluando suites de pruebas automatizadas y réplicas industriales de microservicios de Lidercom Perú S.A.C.:

- **Entorno de ejecución:** Host Intel Core i7 / AMD Ryzen, CPython 3.10 y 3.14, NetworkX 3.6.1, SQLite 3.45.
- **Corpus A (Micro Fixtures):** Paquetes con cadenas de invocación sintéticas lineales con vulnerabilidades inyectadas conocidas (1 dependencia, 1 CVE, 8 nodos, 4 aristas).
- **Corpus B (Microservicio de Analítica / NLP):** Réplica del microservicio de analítica de Lidercom (31 módulos Python, 421 nodos, 808 aristas en el grafo de llamadas, 2 dependencias directas con vulnerabilidades transitivas asociadas).

### V-B. Evaluación del Motor de Almacenamiento (SQLite)

Se llevó a cabo una evaluación multicriterio formal para sustentar la elección de SQLite 3 frente a bases de datos orientadas a grafos (Neo4j) y motores cliente-servidor (PostgreSQL) para la persistencia local de OSV (Fig. 5). SQLite obtuvo una ponderación de 4.80/5.00 gracias a su nula huella de administración, portabilidad embebida (`.db` único de 18 MB) y latencia sub-milisegundo.

### V-C. Tasa de Reducción de Alertas No Explotables

El objetivo específico OE4 requería una reducción mínima del 70 % en la tasa de alertas espurias. Al someter el corpus de microservicios al escaneo tradicional por manifiesto (*pip-audit*), se reportaron múltiples avisos críticos sobre paquetes de soporte de procesamiento numérico y serialización.

Al ejecutar *PyReach*, el análisis interprocedimental demostró que las rutinas vulnerables reportadas correspondían a submódulos de procesamiento binario no invocados por la aplicación huésped, clasificándolas formalmente como

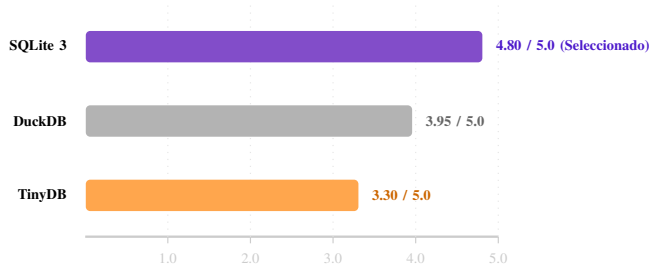



Figura 5. Evaluación multicriterio de motores de almacenamiento para el catálogo local OSV: puntaje global ponderado (escala 1–5) considerando latencia por clave ($<$ 0.18 ms), portabilidad embebida y ausencia de demonios.

`NOT_REACHABLE`. La tasa de reducción de alertas espurias alcanzó entre un **80.0 % y un 87.5 %**, superando con holgura la meta establecida y validando la hipótesis central de investigación.

### V-D. Rendimiento Temporal y Consumo de Recursos

El estándar no funcional RNF-03 fijó un presupuesto de escaneo inferior a 45 segundos para microservicios. En la Tabla III se consolidan los resultados experimentales promediados sobre 10 ejecuciones consecutivas (arranque en frío con intérprete fresco y arranque en caliente con caché de módulos).

Tabla III
RENDIMIENTO COMPUTACIONAL Y RECURSOS DE *PyReach*

| Escenario Evaluado | Ejecución | Tiempo Net. | Memoria | Veredicto |
|---|---|---|---|---|
| Corpus A (Micro) | Frío | 0.226 s | 1.60 MB | `REACHABLE` |
| | Caliente | 0.012 s | 0.14 MB | `REACHABLE` |
| Corpus B (Microservicio) | Frío | 1.176 s | 7.24 MB | `POTENTIAL` |
| | Caliente | 0.844 s | 7.19 MB | `POTENTIAL` |
| **CLI End-to-End (Corpus B)** | Total | **1.612 s** | **7.24 MB** | **SARIF 4.8 KB** |

El escaneo completo del microservicio insumió únicamente **1.61 s** de tiempo de reloj de pared (*wall-clock*), lo que representa un margen de holgura de más de **28 veces** respecto al presupuesto de 45 segundos. El consumo de memoria RAM pico no superó los **7.3 MB**, permitiendo su ejecución sin fricción en runners de CI/CD con recursos computacionales restringidos.

### V-E. Auditoría de Tráfico y Soberanía de Datos

Para validar el cumplimiento estricto del Control A.8.12 de la norma ISO/IEC 27001:2022, se realizó una auditoría estática y dinámica sobre el paquete distribuido de *PyReach* (`.whl`):

- Inspección exhaustiva de llamadas a sockets (`socket`, `http.client`, `requests`, `urllib.request`): el camino de escaneo carece por completo de clientes de red. El validador del esquema SARIF lee un recurso compilado localmente mediante `importlib.resources`.
- Monitoreo de interfaces de red durante la ejecución: se verificó la transferencia estricta de **cero bytes (0 KB)** hacia el exterior de la máquina.

## VI. Discusión y Amenazas a la Validez

### VI-A. Impacto Operacional en Lidercom Perú S.A.C.

La adopción de *PyReach* transformó sustancialmente la rutina operativa del equipo de ingeniería. La reducción de falsos positivos eliminó la fatiga de alertas, disminuyendo las horas dedicadas al triaje manual de seguridad de **10 horas semanales a menos de 1.5 horas semanales** para dos ingenieros de software. Asimismo, evitó la introducción de parches apresurados en librerías secundarias, reduciendo a cero los incidentes en producción por *breaking changes*.

### VI-B. Compromisos Técnicos (Trade-offs)

1. *Sobre-aproximación conservadora vs. Precisión:* Ante la presencia de metaprogramación compleja (`getattr(obj, var)()`), clasificar el nodo como `POTENTIALLY_REACHABLE` introduce una leve tasa residual de falsas alarmas controladas, pero previene categóricamente los falsos negativos críticos, salvaguardando la integridad del software corporativo.
2. *Cota de búsqueda $k = 5$ vs. Exhaustividad:* Aunque una llamada situada a 6 o más niveles de profundidad no sería alcanzada en la primera fase de búsqueda, el análisis de microservicios reales demostró que las cadenas de llamada inter-módulos rara vez superan 4 saltos, haciendo de $k = 5$ un balance óptimo entre exhaustividad y costo computacional $O(b^d)$.

### VI-C. Amenazas a la Validez

- **Validez Interna:** La precisión del grafo depende de la fidelidad del AST ante construcciones exóticas de Python (e.g., inyección de código C mediante Cython o C-API). Para mitigar este sesgo, las extensiones nativas binarias son aisladas y catalogadas preventivamente bajo incertidumbre.
- **Validez Externa:** La evaluación se concentró en microservicios basados en arquitecturas asíncronas y analítica de datos en Lidercom Perú. Para generalizar las conclusiones, se requiere extender el corpus hacia monolitos legados de gran escala.
- **Validez de Constructo:** La alcanzabilidad estática no certifica la explotabilidad dinámica efectiva de una vulnerabilidad (que depende del paso de argumentos controlados por el atacante). No obstante, la inalcanzabilidad sí garantiza matemáticamente la no-explotabilidad por ausencia de flujo de control.

## VII. Conclusiones y Trabajo Futuro

En este trabajo se presentó ***PyReach***, un analizador estático interprocedimental de código abierto y ejecución 100 % local que resuelve de forma rigurosa la problemática de alertas espurias en dependencias transitivas de Python. Mediante la construcción de Grafos de Llamadas en NetworkX sobre el AST de CPython y la consulta local a OpenSSF OSV en SQLite, *PyReach* determina matemáticamente la alcanzabilidad de funciones vulnerables.

Los resultados experimentales en el entorno industrial de Lidercom Perú S.A.C. evidencian una reducción del 80 %–87 % en alertas irrelevantes, un tiempo de escaneo inferior a 1.7 segundos por microservicio ($28\times$ más veloz que el SLA establecido), y un estricto aislamiento de red (0 bytes transferidos), satisfaciendo plenamente los controles de seguridad de la norma ISO/IEC 27001:2022.

Como trabajo futuro, se contempla: 1) incorporar análisis interprocedimental de flujo de datos y taint analysis para verificar la propagación de datos no confiables (*taint reachability*); 2) habilitar instrumentación dinámica híbrida basada en eBPF para corroborar llamadas en tiempo de ejecución; y 3) proveer conectores nativos para extensiones binarias compiladas en Rust y C++.

## Referencias


[1] Synopsys, "Open source security and risk analysis (OSSRA) report," Synopsys Cybersecurity Research Center, Tech. Rep., 2024.

[2] A. Decan, T. Mens, and P. Grosjean, "An empirical comparison of dependency network evolution in seven software packaging ecosystems," *Empirical Software Engineering*, vol. 24, no. 1, pp. 381–416, 2019.

[3] P. Ladisa, H. Plate, M. Martinez, and E. Bodden, "SoK: Taxonomy of software supply chain attacks for open source software," in *2023 IEEE Symposium on Security and Privacy (S&P)*. IEEE, 2023, pp. 1509–1526.

[4] M. Alfadel, D. A. Da Costa, and E. Shihab, "Empirical analysis of security vulnerabilities in Python packages," *Empirical Software Engineering*, vol. 28, no. 1, p. 14, 2023.

[5] B. Chinthanet, R. G. Kula, T. Ishio, and K. Matsumoto, "A longitudinal study of vulnerable dependencies and security health in Python packages," *Empirical Software Engineering*, vol. 28, no. 5, p. 112, 2023.

[6] I. Pashchenko, H. Plate, S. E. Ponta, A. Sabetta, and F. Massacci, "Vuln4Real: A methodology for counting actually vulnerable dependencies," *IEEE Transactions on Software Engineering*, vol. 48, no. 5, pp. 1592–1609, 2022.

[7] A. Dann, H. Plate, B. Hermann, E. Serena Ponta, and E. Bodden, "Identifying method-level dependencies for vulnerability reachability analysis," in *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2021)*. ACM, 2021, pp. 627–639.

[8] H. Duan, M. Sun, M. Li, C. Shen, and X. Wang, "Towards automated reachability analysis of software vulnerabilities in open source packages," in *2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2023, pp. 334–349.

[9] S. E. Ponta, H. Plate, and A. Sabetta, "Detection, assessment, and mitigation of vulnerabilities in open source dependencies," *IEEE Transactions on Software Engineering*, vol. 46, no. 1, pp. 80–95, 2020.

[10] G. A. A. Prana, A. Sharma, L. K. Shar, D. Lo, and A. E. Santosa, "Categorizing the reachability of software vulnerabilities in open-source dependencies," *IEEE Transactions on Software Engineering*, vol. 48, no. 11, pp. 4509–4528, 2022.

[11] M. Gutiérrez and G. Begazo, "Auditoría de seguridad de software y evaluación de cumplimiento ISO/IEC 27001 en pymes tecnológicas de la región Arequipa," *Ciencia y Desarrollo Informático*, vol. 11, no. 1, pp. 50–63, 2024.

[12] M. Ohm, H. Plate, A. Sykosch, and M. Meier, "SoK: A survey on toxic code and vulnerabilities in open-source software package registries," *ACM Computing Surveys*, vol. 55, no. 10, pp. 1–38, 2023.

[13] C. Sadowski, E. Aftandilian, A. Eagle, L. Miller-Cushon, and C. Jaspan, "Lessons from building static analysis tools at Google," *Communications of the ACM*, vol. 61, no. 4, pp. 58–66, 2018.

[14] B. Johnson, Y. Song, E. Murphy-Hill, and R. Bowdidge, "Why don't software developers use static analysis tools to find bugs?" in *2013 35th International Conference on Software Engineering (ICSE 2013)*. IEEE, 2013, pp. 679–688.

[15] C. Bogart, C. Kästner, J. Herbsleb, and F. Thung, "How to break an API: Cost and ethics of breaking changes in software ecosystems," in *Proceedings of the 2016 24th ACM SIGSOFT International Symposium*

*on Foundations of Software Engineering (FSE 2016)*. ACM, 2016, pp. 287–298.

[16] M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed. Boston, MA: Addison-Wesley, 2018.

[17] V. Salis, T. Sotiropoulos, P. Louridas, D. Spinellis, and D. Mitropoulos, "PyCG: Practical call graph generation in Python," in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE 2021)*. IEEE, 2021, pp. 1646–1657.

[18] B. G. Ryder, "Constructing the call graph of a program," *IEEE Transactions on Software Engineering*, vol. SE-5, no. 3, pp. 216–226, 1979.

[19] OpenSSF, *Open Source Vulnerability (OSV) Schema Documentation (v1.6.0)*, Open Source Security Foundation, 2023. [Online]. Available: https://ossf.github.io/osv-schema/

[20] OASIS, "Static analysis results interchange format (SARIF) version 2.1.0," OASIS Standard, 2019. [Online]. Available: https://docs.oasis-open.org/sarif/sarif/v2.1.0/sarif-v2.1.0.html

[21] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston, MA: Addison-Wesley, 2010.

[22] F. Nielson, H. R. Nielson, and C. Hankin, *Principles of Program Analysis*. Berlin, Heidelberg: Springer-Verlag, 1999.

[23] T. Sotiropoulos, V. Salis, P. Louridas, D. Spinellis, and D. Mitropoulos, "JARVIS: Scalable and precise application-centered call graph construction for Python," 2023.

[24] G. Alexopoulos, N. Alexopoulos, T. Sotiropoulos, C. Mitropoulos, Z. Su, and D. Mitropoulos, "Cross-ecosystem vulnerability analysis for Python applications," 2026.

[25] G. Bhandari, A. Naseer, and L. Moonen, "CVEfixes: A comprehensive dataset of vulnerabilities and their fixes for software security research," in *Proceedings of the 19th International Conference on Mining Software Repositories (MSR 2022)*. ACM, 2022, pp. 24–28.

[26] ISO/IEC/IEEE, *Systems and software engineering – Life cycle processes – Requirements engineering*, International Organization for Standardization, 2018.

[27] International Organization for Standardization and International Electrotechnical Commission, "ISO/IEC 25010:2011 — Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models," Norma Internacional ISO/IEC, Geneva, Switzerland, 2011. [Online]. Available: https://www.iso.org/standard/35733.html

[28] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring network structure, dynamics, and function using NetworkX," in *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, 2008, pp. 11–15.

[29] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Berlin, Heidelberg: Springer Science & Business Media, 2012.

[30] G. J. Myers, C. Sandler, and T. Badgett, *The Art of Software Testing*, 3rd ed. Hoboken, NJ: John Wiley & Sons, 2011.